1. DATOS GENERALES

- **Tema:** Proyecto Segundo Parcial
- **Unidad de Organización Curricular:** Profesional
- **Nivel y Paralelo:** Cuarto “A”
- **Alumnos participantes:** Herrerra Lescano Mateo Renato
  Molina Alvarado Anahi Dayana
  Moyolema Criollo Katherine Lissette
  Ortiz Cunalata Alina Elizabeth
  Paredes Claudio Alisson Betzabe
  Zurita Lopez Julio Cesar

- **Asignatura:** Manejo y configuración del software
- **Docente:** Ing. MSc. Santiago David Jara Moya
- **Fecha de entrega:** 25/11/2026

---

2. OBJETIVOS

2.1. General

Aplicar conocimientos sobre Git y plataformas de repositorio remoto mediante la simulación de un entorno real de desarrollo colaborativo, utilizando el modelo GitFlow.

2.2. Específicos

- [Objetivo 1]
- [Objetivo 2]
- [Objetivo 3 – Máximo 3]

---

3. ACTIVIDADES DESARROLLADAS

3.1 Introducción

MoviPass es la aplicación web que el equipo desarrolla en el segundo parcial de **Manejo y Configuración de Software**. Permite a las cooperativas de transporte administrar sus buses, frecuencias y hojas de ruta, y a los usuarios buscar viajes, elegir asientos, pagar y obtener un boleto con código QR. En este parcial lo que se evalúa no es tanto la calidad del código sino cómo se gestiona su evolución: en la primera mitad, el **control de versiones**, y en la segunda, el **control de cambios**.

Este informe documenta cómo el equipo organizó el trabajo con Git y GitHub (flujo Gitflow, Pull Requests con revisión de un comité), qué documentación contiene el repositorio, cómo se gestionan las solicitudes de cambio con un proceso inspirado en ITIL, cómo se audita el trabajo mediante reportes y trazabilidad, y qué aplicación se ha construido hasta ahora. Cada afirmación se respalda con evidencias del repositorio (commits, issues, Pull Requests y capturas de pantalla).

3.2 El proyecto MoviPass

3.2.1 Descripción y alcance

MoviPass gestiona la venta de pasajes de bus interprovincial. Una cooperativa registra sus buses (con la estructura de asientos de cada uno), define frecuencias aprobadas y genera hojas de ruta. El usuario se registra, busca viajes por origen, destino y fecha, selecciona los asientos, ingresa los datos de los pasajeros y paga. El pago puede ser en efectivo (en oficina) o por depósito o transferencia: en este último caso se sube el comprobante, el oficinista lo valida y recién entonces se genera el boleto con QR. El conductor valida los boletos desde su teléfono al iniciar el viaje y puede cobrar a pasajeros que suben en el camino.

3.2.2 Roles del sistema

Rol	Función
Administrador	Crea cooperativas y su usuario gerente.
Cooperativa	Gestiona buses, plantillas de asientos, frecuencias, rutas y precios base.
Oficinista	Vende boletos en oficina y aprueba los comprobantes de pago.
Usuario final (cliente)	Busca viajes, elige asientos, compra y consulta su historial de boletos.
Usuario bus (chofer)	Inicia el viaje, valida boletos, cobra en ruta y obtiene el reporte de pasajeros.

3.2.3 Tecnologías

Capa	Tecnología
Frontend	HTML5, CSS3, JavaScript y Bootstrap 5
Backend como servicio	Supabase (PostgreSQL, Auth y Storage) con acceso mediante supabase-js
Migraciones de base de datos	Supabase CLI (archivos versionados en **supabase/migrations/**)
Código QR	Librería JavaScript **qrcode**
Tareas programadas y notificaciones	pg_cron (Supabase) y tabla de notificaciones
Publicación del frontend	Netlify, solo desde la rama **main**
Control de versiones y de cambios	Git, GitHub (ramas, Pull Requests, Issues, Projects)

3.2.4 Equipo y distribución del trabajo

Integrante	Rol de gestión	Módulo del Sprint 1 (issues)
Katherine Moyolema	Líder · Desarrollo · Comité de cambios	Documentación (#11, #12) módulo chofer
Anahi Molina	Desarrollo · Comité de cambios	Compra, pago y boleto (#44 a #49)
Alisson Paredes	Documentación · Desarrollo	Autenticación y roles (#20 a #25)
Alina Ortiz	Documentación · Desarrollo	Cooperativas y buses (#26 a #31)
Mateo Herrera	Documentación · Desarrollo	Frecuencias y hojas de ruta (#32 a #37)
Julio Zurita	Documentación · Desarrollo	Búsqueda y asientos (#38 a #43)

3.2.5 Cumplimiento de los requisitos del parcial

La tabla resume los requisitos indicados para el proyecto y cómo se cumplen en el repositorio.

Requisito	Cómo se cumple	Estado
README, CONTRIBUTING, seguridad, código de conducta y licencia	README.md, CONTRIBUTING.md, SECURITY.md, CODE_OF_CONDUCT.md y LICENSE (GPL v3)	Cumplido
Otras documentaciones del proyecto	CHANGELOG, CITATION.cff, SUPPORT, FLUJO_GIT, CONTROL_DE_CAMBIOS, DECISIONES, MODELO_DATOS y MIGRACIONES	Cumplido
Herramienta de issues para el control de cambios	Issues de GitHub (decisión D-004), con tres plantillas	Cumplido
Flujo de trabajo con ramas, justificado	Gitflow (docs/FLUJO_GIT.md y sección 4.2)	Cumplido
Comité de control de cambios de 1 o 2 miembros	Katherine Moyolema y Anahi Molina	Cumplido
Plantillas de issues y de Pull Requests en .github	3 plantillas de issues, plantilla de PR y CODEOWNERS	Cumplido
Citado del repositorio	CITATION.cff	Cumplido
Commits calificados desde el 12 de octubre de 2026	Primer commit el 12-oct-2026 a las 19:49	Cumplido
Reportes de auditoría con dashboard y rango de fechas	GitHub Projects con Insights (sección 7)	Por evidenciar
Informe con capturas (documentación y auditoría)	Este documento	En curso
Aplicación con errores hasta el 4 de noviembre	Desarrollo en curso (sección 8)	En curso

3.3 Control de versiones

3.3.1 Herramientas

El código, la documentación y las migraciones de la base de datos viven en un único repositorio de GitHub (**kmoyolema3929/MoviPass**). Se trabaja con Git desde la terminal y desde VS Code. La rama **main** alimenta el sitio publicado en Netlify; el trabajo diario ocurre en **develop** y en ramas de funcionalidad.

3.3.2 Flujo de trabajo elegido: Gitflow

El equipo adoptó **Gitflow** porque separa lo estable (**main**) de lo que está en desarrollo (**develop**), permite que seis personas trabajen en paralelo en ramas **feature/*** sin pisarse, y ofrece un camino definido para preparar versiones (**release/***) y para corregir fallos urgentes (**hotfix/***). Esta justificación consta en **docs/FLUJO_GIT.md**.

Rama	Se crea desde	Se fusiona hacia	Uso
main	—	—	Permanente. Solo versiones estables; al fusionar, Netlify publica el sitio.
develop	main	—	Permanente. Rama predeterminada; integra el trabajo (decisión D-005).
feature/*	develop	develop	Una rama por tarea o issue. Se conserva tras fusionar.
release/*	develop	main y develop	Preparación de una versión. Se conserva tras fusionar.
hotfix/*	main	main y develop	Corrección urgente (cambio de emergencia). Se conserva tras fusionar.

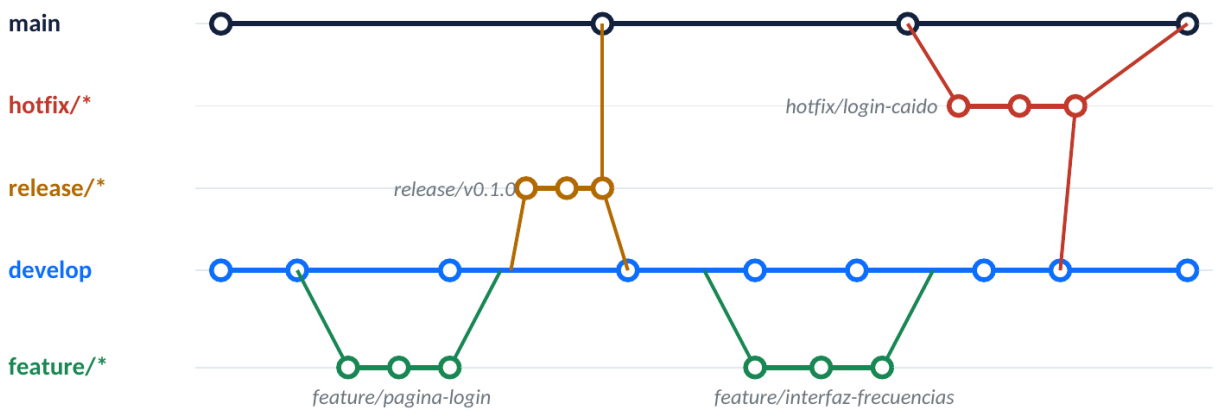


Figura 1. *Esquema de ramas de Gitflow aplicado en MoviPass (ejemplos con ramas reales del proyecto).*

Rama predeterminada

Se eligió **develop** como rama predeterminada del repositorio. GitHub solo cierra automáticamente los issues enlazados con **Closes #n** cuando el Pull Request se fusiona en la rama predeterminada; con develop, los issues se cierran al integrar el trabajo, sin tener que tocar **main** (decisión D-005).

3.3.3 Ciclo de trabajo de un cambio

- Se toma un issue asignado y se actualiza **develop**.
- Se crea la rama de trabajo desde develop, por ejemplo `feature/pagina-login`.
- Se hacen commits pequeños con el formato Conventional Commits y se sube la rama.
- Se abre un Pull Request hacia develop con la plantilla y la línea `Closes #n`. GitHub pide la revisión al comité mediante CODEOWNERS.
- El comité revisa, comenta y aprueba. Nadie aprueba su propio PR.
- El autor responde los comentarios y fusiona con *Create a merge commit*; el issue se cierra solo.
- Cuando develop reúne lo necesario, se crea `release/vX.Y.Z`, se fusiona a main y de vuelta a develop, y se etiqueta la versión.

3.3.4 Convención de ramas y de commits

Las ramas se nombran con el prefijo de su tipo y un nombre corto en minúsculas (`feature/crud-buses`, `feature/manual-usuario`, `hotfix/login-caido`). Los commits siguen **Conventional Commits** con ámbito, en la forma `tipo(ámbito): descripción` (decisión D-007), de modo que el historial se lee igual que las etiquetas de los issues: tipo y módulo.

Tipo	Uso	Ejemplo real del repositorio
`feat`	Nueva funcionalidad	`feat(autenticacion): maquetar la pagina de login con bootstrap`
`fix`	Corrección de error	Previsto desde el control de cambios (después del 4 de noviembre)
`docs`	Documentación	`docs(decisiones): registrar decisiones D-006 y D-007`
`refactor`	Cambio interno sin alterar el comportamiento	`refactor(frecuencias): reemplaza alert por validacion visual was-validated de bootstrap`
`test`	Pruebas	`test(frecuencias): incorpora validaciones requeridas en campos del formulario`
`chore`	Mantenimiento y configuración	`chore(supabase): configura archivo toml para entorno local`

Los commits anteriores a la decisión D-007 (12 a 14 de octubre) usan la forma `tipo: descripción`, sin ámbito; por acuerdo del equipo no se reescriben para conservar el historial real. El pie del commit lleva `Closes #n` cuando resuelve un issue, como en `docs(documentación): crear el modelo de datos y sus convenciones Closes #10`.

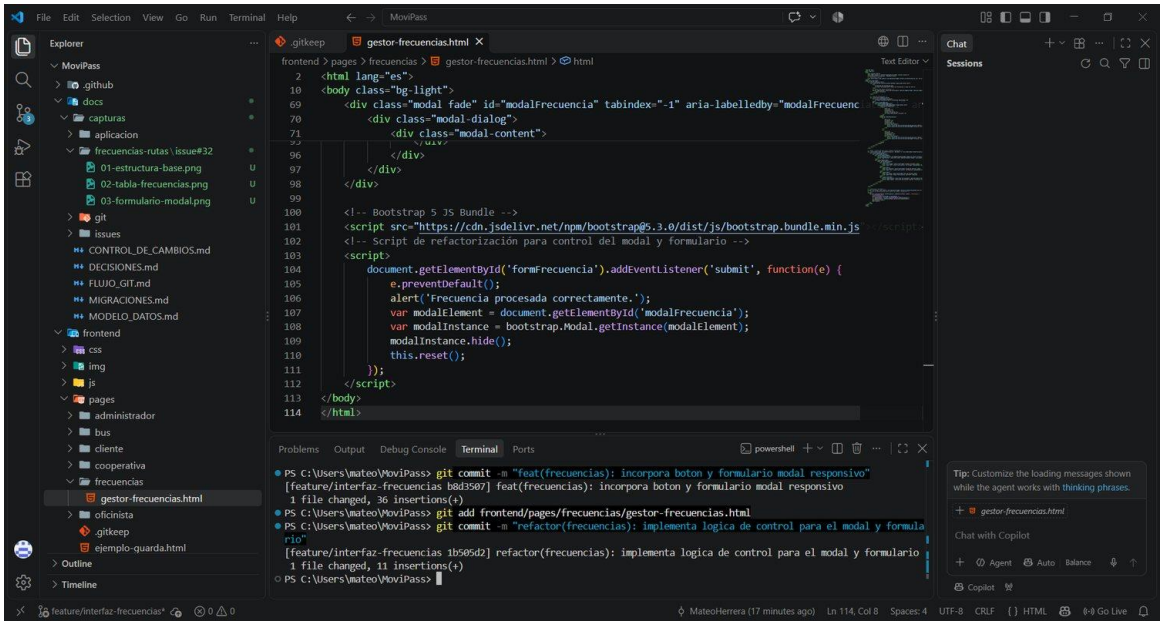


Figura 1. *Terminal de VS Code con commits en formato Conventional Commits en la rama `feature/interfaz-frecuencias` (issue #32, Mateo Herrera).*

3.4.5 Pull Request, revisión y protección de ramas

Todo el código, las migraciones y las pantallas entran por Pull Request. La única excepción es la de la líder del proyecto, que puede subir directamente a develop los cambios de la documentación inicial (decisión D-005). Los PR hacia develop y main los aprueba el **comité de control de cambios** (Katherine Moyolema y Anahi Molina); basta la aprobación de una de las dos y nadie aprueba su propio PR.

Regla (según docs/FLUJO_GIT.md)	main	develop
Pull Request obligatorio, sin push directo	Sí	Sí (excepción de la líder en documentación inicial)
Aprobaciones requeridas	1	1
Aprobación de un code owner (Katherine o Anahi)	Sí	Sí
Descartar aprobaciones al subir commits nuevos	Sí	Sí
Conversaciones resueltas antes de fusionar	Sí	Sí
Force push y borrado de la rama	Prohibidos	Prohibidos

El método de fusión es **Create a merge commit**, que conserva el historial completo de commits de cada persona; no se usan *Squash* ni *Rebase*. Quien revisa deja un comentario con lo revisado, aunque no tenga observaciones, como evidencia. Las ramas feature, release y hotfix **no se borran** al fusionar, para conservar el trabajo como evidencia. La plantilla de PR incluye una lista de verificación (rama, destino, título, `Closes #n`, etiquetas, pruebas, secretos, migraciones y CHANGELOG) y una sección de revisión.

3.3.6 Versionado semántico y registro de cambios

Las versiones siguen el **Versionado Semántico** (MAJOR.MINOR.PATCH) y el archivo **CHANGELOG.md** sigue el formato *Keep a Changelog*, con las secciones Agregado, Cambiado y Eliminado. A la fecha de corte el repositorio no tiene etiquetas (tags) de versión: el CHANGELOG mantiene la sección «Sin publicar» y la primera versión se creará con una rama `release/*` siguiendo el flujo descrito.

3.3.7 Estadísticas del repositorio

3.4 Documentación del repositorio

3.4.1 Investigación: por qué importa la documentación

En la gestión de la configuración del software, la documentación no es un añadido: forma parte de los elementos de configuración que se identifican, se controlan y se auditan junto con el código. Una auditoría física compara lo que realmente se construyó con la documentación técnica que lo describe, y una auditoría funcional comprueba que el elemento cumple lo especificado y que sus documentos de operación y soporte están completos (IEEE 828-2012; NASA SWEHB; CMMI CM.SP 3.2). Dicho de otra forma: un repositorio sin documentación no se puede auditar, porque no hay contra qué comparar.

En GitHub, un conjunto de archivos estándar comunica cómo usar el proyecto, cómo colaborar, cómo reportar problemas y bajo qué licencia se distribuye: README, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY, SUPPORT, LICENSE y las plantillas de issues y de Pull Requests. Junto a ellos, los proyectos con proceso formal añaden documentos de decisiones, de diseño y de cambios.

Categoría	Pregunta que responde	Documentos en MoviPass
Presentación y uso	¿Qué es y cómo se ejecuta?	README.md · GUIA_INSTALACION.md (pendiente)
Colaboración	¿Cómo contribuyo y cómo nos comportamos?	CONTRIBUTING.md · CODE_OF_CONDUCT.md · SUPPORT.md
Seguridad y legal	¿Cómo se protege y bajo qué licencia?	SECURITY.md · LICENSE (GPL v3) · CITATION.cff
Proceso	¿Cómo se trabaja y cómo se cambia?	docs/FLUJO_GIT.md · docs/CONTROL_DE_CAMBIOS.md · plantillas de .github · CODEOWNERS
Técnica y de diseño	¿Cómo está diseñado y por qué?	docs/MODELO_DATOS.md · docs/MIGRACIONES.md · docs/DECISIONES.md
Historial	¿Qué cambió y cuándo?	CHANGELOG.md · historial de Git
Usuario final	¿Cómo se usa la aplicación?	Manual de usuario (issue #12, pendiente)

3.4.2 Inventario de documentos

Archivo	Propósito	Contenido destacado
README.md	Presentación del proyecto	Descripción, roles, tecnologías, instalación, estructura de carpetas, equipo, licencia y cómo citar
CONTRIBUTING.md	Guía de contribución	Regla principal (ningún cambio sin issue), Gitflow, ciclo de trabajo, Conventional Commits, aprobación de PR, cambios en la base de datos
CODE_OF_CONDUCT.md	Convivencia del equipo	Conductas esperadas e inaceptables
SECURITY.md	Seguridad	Cómo reportar vulnerabilidades (respuesta en 72 horas), manejo de claves, RLS obligatorio y datos personales
SUPPORT.md	Soporte	A dónde acudir según el tipo de duda o problema
LICENSE	Licencia	GNU GPL v3
CITATION.cff	Citado del repositorio	Título, autores, licencia GPL-3.0-only y dirección del repositorio
CHANGELOG.md	Historial de cambios	Formato Keep a Changelog con Agregado, Cambiado y Eliminado
.github/ISSUE_TEMPLATE/	Plantillas de issues	Nueva funcionalidad, Reporte de error y Solicitud de cambio; se bloquean los issues en blanco y se enlaza el reporte privado de vulnerabilidades
.github/pull_request_template.md	Plantilla de PR	Descripción, issue relacionado, tipo de cambio, lista de verificación y sección de revisión
.github/CODEOWNERS	Responsables de revisión	Katherine Moyolema y Anahi Molina como code owners
docs/FLUJO_GIT.md	Flujo de ramas	Gitflow, diagrama, aprobación de PR, protección de ramas y método de fusión
docs/CONTROL_DE_CAMBIOS.md	Proceso de cambios	Comité, reglas de decisión, tipos de cambio, estados, etiquetas y auditoría
docs/DECISIONES.md	Registro de decisiones	Nueve decisiones con contexto, motivos, alternativas descartadas y consecuencias
docs/MODELO_DATOS.md	Convenciones de base de datos	Entidades, nombres, tipos, estados, permisos por rol y RLS
docs/MIGRACIONES.md	Guía de migraciones	Cómo vincular el proyecto y crear migraciones con la Supabase CLI
.env.example · config.example.js	Plantillas de configuración	Variables sin valores reales; los archivos reales están en .gitignore

3.4.3 Registro de decisiones técnicas

docs/DECISIONES.md guarda cada decisión con su fecha, estado, contexto, motivos, alternativas descartadas y consecuencias. Las decisiones no se borran ni se reescriben: cuando una se reemplaza,

cambia de estado y se agrega una nueva. Es el registro que permite explicar, ante una auditoría, por qué el proyecto está como está.

ID	Fecha	Decisión	Estado
D-001	12-oct	Backend con Java y Spring Boot (antes Node.js con Express)	Reemplazada por D-002
D-002	14-oct	Supabase como backend (PostgreSQL, Auth y Storage) con migraciones versionadas	Aprobada
D-003	14-oct	Netlify para publicar el frontend	Reemplazada por D-008
D-004	14-oct	Issues de GitHub como herramienta de gestión de cambios, en lugar de Jira	Aprobada
D-005	14-oct	develop como rama predeterminada; aprobación de PR por un comité de dos; excepción de la líder en documentación inicial	Aprobada
D-006	15-oct	Organización de issues y PR para la auditoría: etiquetas de tipo, prioridad y módulo, y fórmula de títulos	Ajustada por D-007
D-007	15-oct	Conventional Commits con ámbito	Aprobada
D-008	15-oct	Netlify solo desde main, sin Deploy Previews ni Branch deploys, para cuidar los créditos	Aprobada
D-009	15-oct	Opciones de seguridad de Supabase: Data API habilitada, sin exposición automática de tablas y RLS automático	Aprobada

3.4.4 Modelo de datos y migraciones

docs/MODELO_DATOS.md define el diagrama de entidades (13 entidades: perfiles, cooperativas, buses, tipos_bus, asientos, frecuencias, paradas, hojas_ruta, viajes, boletos, pasajeros, pagos y notificaciones), las reglas de nombres (tablas en plural, columnas en singular, claves `id uuid`, claves foráneas `<tabla>_id`, fechas `<evento>_en` y estados con `check`), la matriz de permisos por rol y la obligación de activar RLS en toda tabla nueva.

Ningún cambio de esquema se hace desde el panel web de Supabase: todo pasa por archivos de migración en **supabase/migrations/**, revisados en un Pull Request y nunca editados después de fusionados. A la fecha existen tres migraciones:

Archivo	Issue	Contenido
`20261015120000_crear_tabla_prueba.sql`	#6	Tabla de prueba para validar la cadena de migraciones con la Supabase CLI
`20261016090000_crear_perfiles_y_roles.sql`	#7	Tabla perfiles vinculada a auth.users, trigger de creación de perfil, función rol_actual() y políticas RLS
`20261017090000_exponer_perfiles.sql`	#9	Permisos de uso del esquema y de acceso a perfiles para usuarios autenticados

3.4.5 Seguridad y manejo de secretos

SECURITY.md establece que nunca se suben contraseñas ni claves al repositorio, que la clave `service_role` jamás se usa en el frontend, que toda tabla lleva RLS con políticas por rol, que las contraseñas las gestiona Supabase Auth y que las reglas sensibles (venta de asientos, descuentos, validación de boletos) se ejecutan en la base de datos y no en el navegador. El archivo `.env` y el generado `frontend/js/config.js` están en `.gitignore`; en su lugar se versionan `.env.example` y `config.example.js`. En el despliegue, `scripts/generar-config.sh` crea `config.js` a partir de las variables de entorno de Netlify.

3.4.6 Documentación pendiente

3.5 Control de cambios

3.5.1 Fundamento: ITIL y la gestión de cambio

ITIL define un **cambio** como la adición, modificación o eliminación de cualquier elemento que pueda afectar, directa o indirectamente, a los servicios de TI. Aunque la práctica se conocía como «gestión de cambios», ITIL 4 la llama **habilitación de cambios** (*change enablement*) y distingue tres tipos: los cambios **estándar** (de bajo riesgo, repetitivos y preautorizados), los **normales** (requieren evaluación de riesgo y autorización) y los de **emergencia** (urgentes, con aprobación acelerada y documentación posterior). Quien autoriza cada tipo es la *autoridad de cambios*, que puede ser una persona, un equipo, un comité asesor de cambios (CAB) o incluso un mecanismo automatizado; para emergencias suele existir un grupo reducido disponible con poco aviso (ECAB).

MoviPass adapta esta idea a un equipo pequeño: la solicitud se registra como issue de GitHub, un **comité de dos personas** analiza si el cambio es viable, lo aprueba o lo rechaza con justificación, un desarrollador lo implementa en una rama y el comité verifica el resultado antes de cerrarlo. Como resumió el docente: análisis, aprobación del comité y recién entonces el desarrollador.

3.5.2 Comité de control de cambios

Miembro	Rol en el comité
Katherine Moyolema	Miembro del comité y líder del proyecto; administra el repositorio, las ramas protegidas y los releases
Anahi Molina	Miembro del comité

Funciones: evaluar cada solicitud, analizar impacto y riesgo sobre los demás módulos, decidir (aprobar, rechazar, cancelar o pedir más información), priorizar, asignar al responsable, autorizar los cambios de emergencia, aprobar los Pull Requests hacia develop y main, verificar el cambio implementado antes de cerrar el issue, justificar por escrito cada rechazo o cancelación y mantener el tablero y los reportes.

Reglas de decisión: decide una de las dos personas del comité; nadie aprueba, rechaza ni verifica un cambio propio (si una miembro es la solicitante, decide la otra); toda decisión queda en el issue con fecha, responsable y justificación; y un cambio no se implementa sin estado *Aprobado*.

3.5.3 Tipos de cambio

Tipo	Descripción	Aprobación
Normal	Cambio planificado que sigue todo el flujo	Comité
Emergencia	Corrección urgente de un fallo crítico; se trabaja en una rama hotfix	Una miembro del comité (o la líder si no hay disponibilidad); se documenta después
Estándar	Cambio menor y repetitivo de bajo riesgo (por ejemplo, texto o estilo)	Una miembro del comité

3.5.4 Flujo y estados de una solicitud

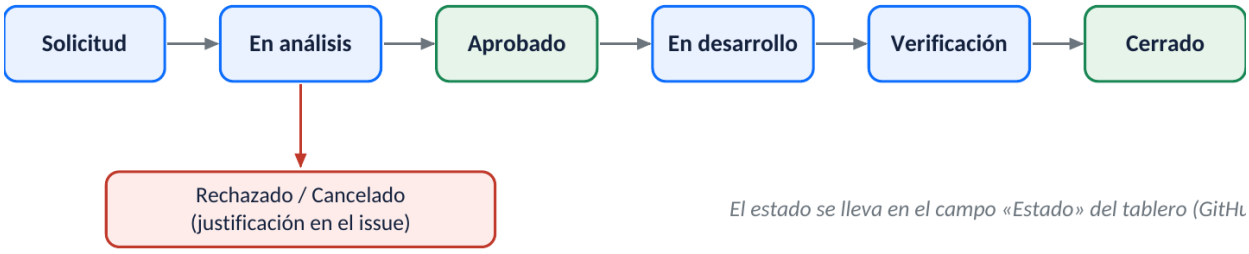


Figura 6. *Estados de una solicitud de cambio en MoviPass.*

Estado	Responsable	Descripción
Solicitud	Solicitante	Se registra con la plantilla «Solicitud de cambio»
En análisis	Comité	Se evalúa la necesidad, el impacto y el riesgo
Aprobado	Comité	Se autoriza, se prioriza y se asigna a quien lo desarrolla
Rechazado	Comité	No procede; se explica el motivo en el issue
En desarrollo	Responsable asignado	Se implementa en una rama vinculada al issue
Verificación	Comité	Se comprueba que funciona y que no rompe otros módulos
Cerrado	Comité	Cambio completado con éxito
Cancelado	Comité o solicitante	Se detiene; se explica el motivo en el issue

3.5.5 La solicitud de cambio

La plantilla .github/ISSUE_TEMPLATE/solicitud-cambio.md pide: solicitante, tipo de cambio (normal, emergencia o estándar), prioridad, descripción, justificación, componentes afectados, análisis de impacto y riesgo (bajo, medio o alto), plan de pruebas, plan de reversa, la decisión

del comité (con quién, cuándo y por qué), los datos de implementación (rama y Pull Request) y la verificación final por el comité.

3.5.6 Títulos y etiquetas para la auditoria

Para que los reportes sean fiables, todo issue y todo Pull Request cumple las mismas reglas (decisión D-006). El título sigue la fórmula `[TIPO] Módulo: verbo en infinitivo y qué se hace`, por ejemplo `[DOCS] Documentación: crear modelo de datos`, y coincide siempre con las etiquetas.

Grupo (una de cada uno)	Etiquetas
Tipo	`feature`, `docs`, `chore`, `bug`, `cambio`
Prioridad	baja, media, alta, crítica
Módulo	base, autenticación, cooperativas y buses, frecuencias y rutas, búsqueda y asientos, compra y pago, oficinista y configuración, documentación
Además	Milestone, responsable asignado y, al cerrar, el motivo *Completed* o *Not planned* con un comentario

3.5.7 Trazabilidad entre issue, rama, commit y Pull Request

Cada cambio deja una cadena verificable: el **issue** con criterios de aceptación y rama sugerida → la **rama** `feature/*` → los **commits** con tipo y ámbito → el **Pull Request** con `Closes #n` y las mismas etiquetas → la **aprobación del comité** → el **merge commit** en develop → el **cierre automático** del issue. La matriz de la sección 7.4 muestra esta cadena para los cambios ya integrados.

3.5.8 Cambios gestionados hasta la fecha

3.6 Auditoria y reportes

3.6.1 Investigación: la auditoria de configuración

3.6.2 Reportes y tablero (dashboard)

3.6.3 Metodología de verificación aplicada a este informe

3.6.4 Matriz de trazabilidad: issue, rama y Pull Request

……..

3.7 Aplicación desarrollada

3.7.1 Arquitectura

3.7.2 Estructura del frontend

3.7.3 Base de datos implementada

3.7.4 Módulos implementados

3.7.5 Estado frente a los requisitos

3.8 Cronograma y trabajo pendiente

---

4. RESULTADOS OBTENIDOS

4.1. Trabajo colaborativo

4.2. Revisión de código

4.3. Sitio web obtenido

4.4. Estado final del repositorio

4.5. Problemas encontrados y soluciones

4.6. Cumplimiento de los requisitos

---

5. CONCLUSIONES

[Constituye la respuesta que propone el o los estudiantes para el problema que originó el trabajo desarrollado, de acuerdo con los datos recogidos y la teoría aplicada.]

---

6. RECOMENDACIONES

[Redactar las sugerencias o acciones concretas que se derivan de la guía práctica desarrollada.]

---

7. REFERENCIAS BIBLIOGRÁFICAS

[Insertar las referencias bibliográficas empleadas aplicando la norma IEEE.]

---

8. ANEXOS

[Especifique los entregables adicionales de la Guía, por ejemplo: Codigo fuente, manual de usuario, infografía, etc.]